

TP N2 — Le tokeniseur appris

Module NLP, séance 2 — projet **ILIACode**

Travail Individuel

Pr. Asmaa RASSIL

ENSA Fès, Filière ILIA (3^e année)

Le code couleur du document

Chaque nature d'objet porte sa couleur, la même d'un bout à l'autre du TP.

- **bpe.py** est un fichier ou un dossier.
- **en_octets** est une des fonctions à compléter.
- **pytest** est une bibliothèque ou un objet importé.
- **slide 7** renvoie au cours de la séance.
- **notion importante**, **erreur ou test en échec**, **test au vert**.
- commande tapée ou affichage de l'écran, sans nature particulière.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Les octets
- 3 Partie 2 — Pré-tokeniser
- 4 Partie 3 — Apprendre
- 5 Partie 4 — Encoder
- 6 Partie 5 — Décoder
- 7 Partie 6 — Mesurer
- 8 Partie 7 — Publier

Partie 0 — La mise en place

Lien avec le cours

Cette partie suit les **slides 2** (Le token inconnu) et **3** (Le déroulement de la séance). Elle prépare les fichiers annoncés au **slide 3**.

Étapes 0.1 et 0.2 — Le dossier de la séance

Étape 0.1. Placer le dossier de la séance dans votre dépôt.

Faites un clic droit sur **seance2_etudiants.zip**, choisissez *Extraire tout*, et placez le dossier **seance2** obtenu **à l'intérieur** de votre dossier **iliacode**, à côté de **seance1**.

Étape 0.2. Ouvrir le dépôt dans VS Code.

Lancez VS Code, menu *Fichier, Ouvrir le dossier*, et choisissez **iliacode**. Dans la colonne de gauche, déployez **seance2**. Vous devez y voir **bpe.py**, **test_bpe.py**, **outils.py**, **mesure_fertilite.py** et un dossier **donnees** qui contient **entrainement** et **test**.

Étape 0.3 — Le terminal

Étape 0.3. Ouvrir le terminal et entrer dans le dossier.

Menu *Terminal*, *Nouveau terminal*. Activez votre environnement virtuel

```
.venv\Scripts\activate.bat
```

son nom doit apparaître entre parenthèses au début de la ligne. Puis tapez la commande suivante :

```
cd seance2
```

Aucune bibliothèque nouvelle n'est à installer, **pytest** l'a été au TP N1.

Étape 0.4 — Les 10 tests une première fois

Étape 0.4. Lancer les 10 tests une première fois.

```
pytest -v
```

Vérification avant de continuer

Dix lignes se terminent par **FAILED**, et la dernière ligne de l'écran est la suivante (le temps affiché varie).

```
===== 10 failed in 0.07s =====
```

C'est **normal**. Les fonctions sont encore vides. La séance consiste à faire passer ces 10 lignes au vert.

Étape 0.5 — Le fichier à compléter

Étape 0.5. Regarder le fichier à compléter.

Cliquez sur **bpe.py**. Il contient les fonctions du tableau du **slide 3**. Chaque fonction à écrire se termine par une ligne raise `NotImplementedError("partie N du TP")`, qui dit « pas encore écrit » et nomme la partie du TP où elle s'écrit. Compléter une fonction, c'est **remplacer cette ligne** par le vrai code.

Étape 0.5 — Les trois constantes et la règle du parcours

Trois constantes en haut du fichier servent partout.

- FIN, le marqueur de fin de mot `</w>`.
- BASE, la liste des 256 symboles d'octets.
- MOTIF, l'expression régulière de la pré-tokenisation.

Règle du parcours

Après chaque modification du fichier, enregistrez avec **Ctrl+S** avant de lancer les tests. Dans les cadres gris qui montrent l'interpréteur, ne tapez pas les `>>>`, tapez seulement ce qui vient après.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Les octets**
- 3 Partie 2 — Pré-tokeniser
- 4 Partie 3 — Apprendre
- 5 Partie 4 — Encoder
- 6 Partie 5 — Décoder
- 7 Partie 6 — Mesurer
- 8 Partie 7 — Publier

Partie 1 — Les octets

Lien avec le cours

Cette partie applique le **slide 7** (La base universelle des 256 octets). L'étape 1.1 rejoue les huit octets de معطل du cadre Solution, l'étape 1.3 est l'encadré Application.

Étape 1.1 — Les octets d'un mot

Étape 1.1. Voir les octets d'un mot (slide 7).

Ouvrez l'interpréteur avec la commande python, puis tapez ces lignes. Le mot arabe est écrit par ses numéros Unicode pour éviter de changer de clavier.

```
>>> "valid".encode("utf-8")
b'valid'
>>> len("valid".encode("utf-8"))
5
>>> "\u0645\u0639\u0637\u0644".encode("utf-8")
b'\xd9\x85\xd8\xb9\xd8\xb7\xd9\x84'
>>> len("\u0645\u0639\u0637\u0644".encode("utf-8"))
8
```

Étape 1.1 — Les octets d'un mot (suite)

Cinq lettres ASCII donnent **cinq octets**, quatre lettres arabes donnent **huit octets**, deux par lettre, exactement les D9 85 D8 B9 D8 B7 D9 84 du **slide 7**.

Chaque octet est un nombre entre **0 et 255**. C'est cette borne qui rend la base universelle, tout texte, quelle que soit sa langue, se ramène à ces 256 valeurs.

Étape 1.2 — Un symbole par octet

Étape 1.2. Donner un symbole à chaque octet.

Un symbole de base est le caractère dont le numéro est la valeur de l'octet, `chr(b)`. Pour les octets de l'ASCII, c'est la lettre elle-même. Pour les autres, c'est un caractère quelconque qui sert seulement d'étiquette.

```
>>> [b for b in b'valid']  
[118, 97, 108, 105, 100]  
>>> [chr(b) for b in b'valid']  
['v', 'a', 'l', 'i', 'd']  
>>> chr(0xD9), chr(0x85) Latin by default.  
('Ù', '\x85') caractere de config.
```

Étape 1.3 — La fonction `en_octets`

Étape 1.3. Compléter la fonction `en_octets` (slide 7, encadré Application).

Dans `bpe.py`, trouvez `en_octets`. Remplacez sa ligne raise par la ligne suivante.

```
return [chr(b) for b in mot.encode("utf-8")] + [FIN]
```

Elle encode le mot en UTF-8, remplace chaque octet par son symbole, et ajoute le marqueur de fin de mot. Enregistrez avec **Ctrl+S**.

Étape 1.3 — L'essai dans l'interpréteur

Quittez l'interpréteur avec `exit()` s'il est encore ouvert, relancez python et essayez.

```
>>> from bpe import en_octets
>>> en_octets("valid")
['v', 'a', 'l', 'i', 'd', '</w>']

>>> exit()
```

Étape 1.4 — Les deux tests de la partie

Étape 1.4. Lancer les deux tests de la partie.

```
pytest -v -k octets
```

Vérification avant de continuer

```
test_bpe.py::test_octets_ascii PASSED
test_bpe.py::test_octets_arabe PASSED
===== 2 passed, 8 deselected in 0.03s =====
```

Les deux lignes se terminent par **PASSED**.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Les octets
- 3 Partie 2 — Pré-tokeniser**
- 4 Partie 3 — Apprendre
- 5 Partie 4 — Encoder
- 6 Partie 5 — Décoder
- 7 Partie 6 — Mesurer
- 8 Partie 7 — Publier

Partie 2 — Pré-tokeniser

Lien avec le cours

Cette partie applique le **slide 9** (Le Byte Pair Encoding et ses deux étages). L'étape 2.1 montre la ligne du développeur qui traverse l'étage 1, l'étape 2.2 ajoute le découpage du camelCase annoncé dans le cadre Solution.

Étape 2.1 — L'étage 1 au travail

Étape 2.1. Voir l'étage 1 au travail (slide 9).

La fonction **pretokeniser** est déjà écrite, sauf un geste. Relancez python.

```
>>> from bpe import pretokeniser
>>> pretokeniser("def calculerRistourneCNSS(salaire):")
['def', ' ', 'calculerRistourneCNSS', '(', 'salaire', ')', ':']
>>> pretokeniser("def test_ristourne_plafonnee():")
['def', ' ', 'test', '_', 'ristourne', '_', 'plafonnee', '(', ')', ':']
>>> exit()
```

Étape 2.1 — L'étage 1 au travail (suite)

- **Les blancs sont conservés comme mots**, l'espace entre def et le nom est un mot à part, c'est ce qui rendra le **décodage exact**. Les tirets bas coupent déjà le nom du test.
- En revanche le nom calculerRistourneCNSS est resté entier, **le camelCase n'est pas encore coupé**.

Étape 2.2 — Le découpage du camelCase

Étape 2.2. Ajouter le découpage du camelCase (slide 9).

Dans `pretokeniser`, trouvez la ligne qui porte le commentaire partie 2 du TP.

```
mots.append(partie)
```

Remplacez-la par la ligne suivante, en gardant exactement les mêmes espaces au début.

```
mots.extend(p for p in re.split(CAMEL, partie) if p)
```

Étape 2.2 — Ce que fait la ligne

- CAMEL, défini en haut du fichier, est une expression régulière qui désigne les endroits où une minuscule ou un chiffre est suivi d'une majuscule, et où une suite de majuscules se termine.
- **re.split** coupe le mot à ces endroits.
- **if** p écarte les morceaux vides.

Enregistrez avec **Ctrl+S**.

Étape 2.2 — L'essai

Relancez python.

```
>>> from bpe import pretokeniser
>>> pretokeniser("def calculerRistourneCNSS(salaire):")
['def', ' ', 'calculer', 'Ristourne', 'CNSS', '(', 'salaire', ')', ':']
>>> pretokeniser("t7bes 3.14 !=")
['t7bes', ' ', '3.14', ' ', '!=']
>>> exit()
```

Le nombre à virgule et l'opérateur composé restent entiers, **règles 2 et 3 de la séance 1**, et t7bes aussi, car un chiffre au milieu de minuscules n'est pas un changement de casse.

Étape 2.3 — Le test de la partie

Étape 2.3. Lancer le test de la partie.

```
pytest -v -k pretokeniser
```

Vérification avant de continuer

```
test_bpe.py::test_pretokeniser_camelcase_et_tirets PASSED  
===== 1 passed, 9 deselected in 0.02s =====
```

Lancez ensuite pytest sans option. La dernière ligne doit annoncer **7 failed, 3 passed**.

Partie 2 — Dans une autre application

Dans une autre application

Type d'application. Un assistant de pharmacie, où 500 mg doit rester un mot.

Comment. Changer de domaine, c'est ajouter une alternative à gauche du cœur de MOTIF, exactement comme au TP N1. La ligne suivante, que vous pouvez essayer puis retirer, protège les dosages.

```
MOTIF = r'\d+ *mg|\d+\.\d+|!=|<=|>=|\w+|\s+|[^\\w\\s]'
```

Partie 2 — Dans une autre application (suite)

Dans une autre application

Avec elle, `pretokeniser("prendre 500 mg")` rend `['prendre', ' ', '500 mg']`.

L'étage 2, BPE, n'a rien à savoir du domaine, il reçoit des mots et des frontières. Remettez la ligne d'origine avant de continuer, sinon le test de la partie 2 échoue.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Les octets
- 3 Partie 2 — Pré-tokeniser
- 4 Partie 3 — Apprendre**
- 5 Partie 4 — Encoder
- 6 Partie 5 — Décoder
- 7 Partie 6 — Mesurer
- 8 Partie 7 — Publier

Lien avec le cours

Cette partie applique les **slides 10** (L'algorithme d'entraînement sur un extrait du dépôt) et **11** (Les douze premières fusions et le vocabulaire obtenu). L'étape 3.1 construit l'état initial du tableau du **slide 10**, les étapes 3.2 et 3.3 écrivent les gestes compter et fusionner, l'étape 3.5 écrit la boucle du cadre Solution, l'étape 3.6 retrouve le tableau du **slide 11**.

Étape 3.1 — L'état initial

Étape 3.1. Construire l'état initial (slide 10).

Le mini-corpus est un **Counter**, un dictionnaire qui associe chaque mot à sa fréquence. Relancez python.

```
>>> from collections import Counter
>>> from bpe import en_octets
>>> mini = Counter({"valid": 6, "valide": 3, "valider": 2, "email": 4})
>>> corpus = {tuple(en_octets(m)): f for m, f in mini.items()}
>>> corpus
{('v', 'a', 'l', 'i', 'd', '</w>'): 6,
 ('v', 'a', 'l', 'i', 'd', 'e', '</w>'): 3,
 ('v', 'a', 'l', 'i', 'd', 'e', 'r', '</w>'): 2,
 ('e', 'm', 'a', 'i', 'l', '</w>'): 4}
>>> exit()
```

Étape 3.1 — L'état initial (suite)

C'est le tableau du **slide 10**, chaque mot est devenu sa suite de symboles close par le marqueur, et garde sa fréquence.

La suite est un tuple et non une liste, car **une clé de dictionnaire ne peut pas être modifiable**.

Étape 3.2 — La fonction `compter_paires`

Étape 3.2. Compléter la fonction `compter_paires` (slide 10, geste 1).

Remplacez la ligne raise de `compter_paires` par ces cinq lignes.

```
pairs = Counter()
for symboles, freq in corpus.items():
    for i in range(len(symboles) - 1):
        pairs[(symboles[i], symboles[i + 1])] += freq
return pairs
```

Enregistrez avec **Ctrl+S**.

Étape 3.2 — Ce que font les cinq lignes

- `paires = Counter()` crée le compteur vide des paires.
- La première boucle parcourt chaque mot avec sa fréquence.
- La seconde boucle parcourt les positions `i` du mot, et `(symboles[i], symboles[i + 1])` est la paire de symboles adjacents à cette position.
- `+= freq` ajoute la fréquence du mot et non 1, c'est la **pondération** du **slide 10**, la paire est présente dans chaque occurrence du mot.

Étape 3.2 — L'essai

Relancez python, et reconstruisez corpus comme à l'étape 3.1 avant de compter.

```
>>> from bpe import compter_paires
>>> paires = compter_paires(corpus)
>>> paires.most_common(6)
[ (('v', 'a'), 11), (('a', 'l'), 11), (('l', 'i'), 11),
  (('i', 'd'), 11), (('d', '</w>'), 6), (('d', 'e'), 5) ]
>>> paires[("e", "m")]
4
>>> exit()
```

Ce sont les comptes du **slide 10**, $c(v, a) = 6 + 3 + 2 = 11$, quatre paires ex æquo à 11, puis 6, puis 5, et 4 pour les paires de email.

Étape 3.3 — La fonction `fusionner_mot`

Étape 3.3. Compléter la fonction `fusionner_mot` (slide 10, geste 3).

Remplacez la ligne raise de `fusionner_mot` par ces neuf lignes.

```
a, b = paire
resultat, i = [], 0
while i < len(symboles):
    if i < len(symboles) - 1 and symboles[i] == a and symboles[i + 1] == b:
        resultat.append(a + b)
        i += 2
    else:
        resultat.append(symboles[i])
        i += 1
return tuple(resultat)
```

Étape 3.3 — Ce que fait la fonction

La fonction parcourt le mot **de gauche à droite**. Quand elle voit a suivi de b, elle écrit le symbole collé a + b et saute deux positions. Sinon elle recopie le symbole et avance d'une position.

La fonction **fusionner**, déjà écrite juste en dessous, applique ce geste à tous les mots du corpus. Enregistrez avec **Ctrl+S**.

Étape 3.4 — Les tests des deux gestes

Étape 3.4. Lancer les tests des deux gestes.

```
pytest -v -k "compter or fusionner"
```

Vérification avant de continuer

```
test_bpe.py::test_compter_paires PASSED  
test_bpe.py::test_fusionner_mot PASSED  
===== 2 passed, 8 deselected in 0.02s =====
```

Étape 3.5 — La boucle de **entraîner**

Étape 3.5. Compléter la boucle de la fonction **entraîner** (**slide 10**, cadre **Solution**).

La fonction **entraîner** est déjà écrite autour de sa boucle. Elle construit l'état initial, répète la boucle `nb_fusions` fois, puis assemble `V`. Remplacez la ligne `raise` à l'intérieur de la boucle par ces cinq lignes.

```
    paires = compter_paires(corpus)
    if not paires:
        break
    meilleure = max(paires, key=lambda p: (paires[p], longueur(p[0] + p[1])))
    corpus = fusionner(meilleure, corpus)
    fusions.append(meilleure)
```

Étape 3.5 — Les trois gestes du cours

- **Compter**, la première ligne appelle votre fonction de l'étape 3.2.
- `if not paires`: `break` arrête la boucle si plus aucune paire n'existe, le cas où chaque mot est devenu un seul symbole.
- **Choisir**, `max` retient la paire au plus grand compte, et à compte égal celle dont le symbole collé est le plus long, **longueur** étant déjà écrite. C'est le critère du **slide 10**, et il désigne (`v`, `a`) parmi les quatre paires à 11.
- **Fusionner**, l'avant-dernière ligne applique la fusion à tout le corpus, et la dernière la **consigne** dans la liste ordonnée.

Enregistrez.

Étape 3.6 — Les six premières fusions

Étape 3.6. Retrouver les fusions du cours (slide 11).

Relancez python.

```
>>> from collections import Counter
>>> from bpe import entrainer
>>> mini = Counter({"valid": 6, "valide": 3, "valider": 2, "email": 4})
>>> fusions, V = entrainer(mini, 6)
>>> for a, b in fusions: print((a, b), "->", a + b)
...
('v', 'a') -> va
('va', 'l') -> val
('val', 'i') -> vali
('vali', 'd') -> valid
('valid', '</w>') -> valid</w>
('valid', 'e') -> valide
```

Après la ligne `for`, l'interpréteur affiche et attend, appuyez sur Entrée une seconde

Étape 3.6 — Les douze fusions et le vocabulaire

Dans le même interpréteur, entraînez maintenant douze fusions.

```
>>> fusions, V = entrainer(mini, 12)
>>> [a + b for a, b in fusions]
['va', 'val', 'vali', 'valid', 'valid</w>', 'valide', 'em', 'ema', 'emai',
 'email', 'email</w>', 'valide</w>']
>>> len(V)
269
>>> exit()
```

Les six premières fusions sont celles du tableau du **slide 11**, dans le même ordre. V compte 256 octets, le marqueur et 12 symboles appris, $256 + 1 + 12 = 269$.

Étape 3.7 — Les tests de la partie

Étape 3.7. Lancer les tests de la partie.

```
pytest -v -k "compter or fusionner or entrainer"
```

Vérification avant de continuer

```
test_bpe.py::test_compter_paires PASSED
test_bpe.py::test_fusionner_mot PASSED
test_bpe.py::test_entrainer_douze_fusions PASSED
===== 3 passed, 7 deselected in 0.01s =====
```

Lancez ensuite pytest sans option. La dernière ligne doit annoncer **4 failed, 6 passed**. **La phase 1 est écrite, le vocabulaire s'apprend.**

— Pause de 15 minutes —

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Les octets
- 3 Partie 2 — Pré-tokeniser
- 4 Partie 3 — Apprendre
- 5 Partie 4 — Encoder**
- 6 Partie 5 — Décoder
- 7 Partie 6 — Mesurer
- 8 Partie 7 — Publier

Lien avec le cours

Cette partie applique les **slides 13** (L'encodage par rejeu ordonné des fusions) et **14** (La vérification sur les mots du développeur et du client). L'étape 4.1 écrit l'algorithme du cadre Solution du **slide 13**, l'étape 4.2 rejoue `valids`, l'étape 4.3 le tableau `invalidation` du **slide 14**, l'étape 4.5 le cas `t7bes`.

Étape 4.1 — La fonction `encoder_mot`

Étape 4.1. Compléter la fonction `encoder_mot` (slide 13, cadre Solution).

La fonction reçoit un mot et `rangs`, un dictionnaire qui associe à chaque fusion apprise son rang dans la liste, 0 pour la première, construit par `table_des_rangs`, déjà écrite. Remplacez la ligne raise par ces lignes.

```
symboles = en_octets(mot)
while len(symboles) > 1:
    candidates = [(rangs[(a, b)], (a, b))
                  for a, b in zip(symboles, symboles[1:])]
    if (a, b) in rangs:
        if not candidates:
            break
    _, paire = min(candidates)
    symboles = list(fusionner_mot(paire, symboles))
return symboles
```

Étape 4.1 — Ce que fait la fonction

- La première ligne est l'étape 1 du **slide 13**, le mot en symboles de base clos par le marqueur.
- `zip(symboles, symboles[1:])` parcourt les paires adjacentes du mot, et `candidates` ne garde que celles qui ont été apprises, avec leur rang.
- `if not candidates: break` est l'étape 3, plus aucune fusion ne s'applique.
- `min(candidates)` retient le **rang le plus petit**, la fusion la plus anciennement apprise, c'est l'étape 2, et **fusionner_mot** l'applique.

Enregistrez.

Étape 4.2 — Un mot jamais vu

Étape 4.2. Rejouer sur un mot jamais vu (slide 13).

Relancez python.

```
>>> from collections import Counter
>>> from bpe import entrainer, table_des_rangs, encoder_mot
>>> mini = Counter({"valid": 6, "valide": 3, "valider": 2, "email": 4})
>>> fusions, V = entrainer(mini, 12)
>>> rangs = table_des_rangs(fusions)
>>> rangs[("v", "a")], rangs[("valid", "</w>")]
(0, 4)
>>> encoder_mot("valids", rangs)
['valid', 's', '</w>']
```

Étape 4.2 — La lecture du résultat

Le mot `valids` n'est pas dans le corpus. Les fusions de rang 0 à 3 ont reconstruit `valid`, puis ni `(valid, s)` ni `(s, </w>)` n'ont été apprises, l'encodage s'arrête.

Trois tokens, **aucun inconnu**.

Étape 4.3 — Le mot du développeur

Étape 4.3. Le mot du développeur (slide 14, tableau).

Restez dans le même interpréteur.

```
>>> encoder_mot("invalidation", rangs)
['i', 'n', 'valid', 'a', 't', 'i', 'o', 'n', '</w>']
>>> exit()
```

Neuf tokens, le radical **valid** est reconnu au milieu du mot, et la fusion de rang 4, (valid, </w>), ne s'applique pas car valid est suivi de a, comme le tableau du **slide 14** l'explique.

Étape 4.4 — Les tests de la partie

Étape 4.4. Lancer les tests de la partie.

```
pytest -v -k encoder
```

Vérification avant de continuer

```
test_bpe.py::test_encoder_mots_proches_du_corpus PASSED
test_bpe.py::test_encoder_mot_jamais_vu_sans_inconnu PASSED
===== 2 passed, 8 deselected in 0.02s =====
```

Lancez ensuite pytest sans option. La dernière ligne doit annoncer **2 failed, 8 passed**.

Étape 4.5 — Le mot du client

Étape 4.5. Le mot du client (slide 14, cadre Vérification).

La fonction **encoder**, déjà écrite, enchaîne **pretokeniser** et **encoder_mot** sur un texte entier. Relancez python et refaites les quatre premières lignes de l'étape 4.2, puis tapez ceci.

```
>>> from collections import Counter
>>> from bpe import entrainer, table_des_rangs, encoder_mot, encoder
>>> mini = Counter({"valid": 6, "valide": 3, "valider": 2, "email": 4})
>>> fusions, V = entrainer(mini, 12)
>>> rangs = table_des_rangs(fusions)
>>> encoder_mot("t7bes", rangs)
['t', '7', 'b', 'e', 's', '</w>']
>>> tokens = encoder("l'app t7bes mnin kanseivi le contrat", fusions)
>>> len(tokens)
49
>>> all(t in V for t in tokens)
True
```

Étape 4.6 — La lecture du résultat

Étape 4.6.

- Aucun caractère de t7bes n'apparaît dans le mini-corpus. Ici le mot s'écrit en six octets, tous dans V par construction, aucune fusion ne s'applique, et le résultat est six tokens.
- La dernière ligne vérifie le théorème du **slide 14** sur le ticket entier, **chacun des 49 tokens appartient à V** , il n'y a pas de place pour un inconnu. Le prix est élevé, **49 tokens pour un ticket de huit mots**, parce que le mini-corpus ne contient aucun mot du ticket. **La partie 6 montre ce que le corpus réel change.**

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Les octets
- 3 Partie 2 — Pré-tokeniser
- 4 Partie 3 — Apprendre
- 5 Partie 4 — Encoder
- 6 Partie 5 — Décoder**
- 7 Partie 6 — Mesurer
- 8 Partie 7 — Publier

Lien avec le cours

Cette partie applique le **slide 15** (La réversibilité et le mot du testeur). L'étape 5.1 est l'encadré Application, l'étape 5.2 rejoue l'aller-retour de la Preuve sur les trois textes de la production.

Étape 5.1 — La fonction **decoder**

Étape 5.1. Compléter la fonction **decoder** (slide 15, encadré Application).

Remplacez la ligne raise de **decoder** par ces deux lignes.

```
texte = "".join(tokens).replace(FIN, "")  
return texte.encode("latin-1").decode("utf-8")
```

Enregistrez avec **Ctrl+S**.

- La première ligne colle tous les tokens et retire les marqueurs. La chaîne obtenue est faite de symboles de base, chacun le caractère de numéro *b*.
L'encoder en **Latin-1**, la règle à un octet par caractère de la séance 1, redonne donc exactement les octets d'origine, et les décoder en **UTF-8** redonne le texte.

Étape 5.2 — L'aller-retour

Étape 5.2. Rejouer l'aller-retour (slide 15, Preuve).

Relancez python, refaites les quatre premières lignes de l'étape 4.2, puis tapez ceci.

```
>>> from bpe import encoder, decoder
>>> ticket = "l'app t7bes mnin kanseivi le contrat"
>>> decoder(encoder(ticket, fusions))
"l'app t7bes mnin kanseivi le contrat"
>>> decoder(encoder(ticket, fusions)) == ticket
True
>>> code = "  if valid:\n    return \u0645\u0639\u0637\u0644"
>>> decoder(encoder(code, fusions)) == code
True
>>> exit()
```

Le texte revient à l'identique, espaces compris, et le second essai contient une indentation Python et un mot arabe. C'est **E4, la réversibilité**.

Étape 5.3 — Le test de la partie

Reversibilité garantie.....

Étape 5.3. Lancer le test de la partie.

```
pytest -v -k decoder
```

Vérification avant de continuer

```
test_bpe.py::test_decoder_reversible PASSED  
===== 1 passed, 9 deselected in 0.01s =====
```

Lancez ensuite pytest sans option. La dernière ligne doit annoncer **1 failed, 9 passed**.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Les octets
- 3 Partie 2 — Pré-tokeniser
- 4 Partie 3 — Apprendre
- 5 Partie 4 — Encoder
- 6 Partie 5 — Décoder
- 7 Partie 6 — Mesurer**
- 8 Partie 7 — Publier

Lien avec le cours

Cette partie applique les **slides 17** (La fertilité et son triple coût) et **18** (Le choix de la taille du vocabulaire). L'étape 6.1 écrit la formule encadrée du **slide 17**, l'étape 6.2 retrouve les 97 et 19 tokens du **slide 11**, l'étape 6.4 produit sur votre machine la courbe du **slide 18**, l'étape 6.5 rejoue le ticket du client au vocabulaire retenu.

Étape 6.1 — La fonction `fertilite`

Étape 6.1. Compléter la fonction `fertilite` (slide 17, Définition).

Remplacez la ligne raise de `fertilite` par ces quatre lignes.

```
rangs = table_des_rangs(fusions)
mots = [m for m in pretokeniser(texte) if not m.isspace()]
nb_tokens = sum(len(encoder_mot(m, rangs)) for m in mots)
return nb_tokens / len(mots)
```

La deuxième ligne compte les mots en sortie de l'étage 1, blancs exclus, la troisième compte les tokens que l'étage 2 produit pour ces mots, et la dernière est la **formule encadrée** du slide 17. Enregistrez.

Étape 6.2 — Les chiffres du cours

Étape 6.2. Retrouver les chiffres du cours (slide 11).

Relancez python.

```
>>> from collections import Counter
>>> from bpe import entrainer, fertilite
>>> mini = Counter({"valid": 6, "valide": 3, "valider": 2, "email": 4})
>>> fusions, V = entrainer(mini, 12)
>>> mots = ["valid"] * 6 + ["valide"] * 3 + ["valider"] * 2 + ["email"] * 4
>>> extrait = " ".join(mots)
>>> fertilite(extrait, [])
6.466666666666667
>>> fertilite(extrait, fusions)
1.2666666666666666
>>> exit()
```

Étape 6.2 — La lecture des deux nombres

Sans fusion, l'extrait de 15 mots coûte 97 tokens, $97/15 \approx 6,47$.

Après douze fusions il en coûte 19, $19/15 \approx 1,27$, **la longueur a été divisée par cinq**, c'est la Vérification du **slide 11**.

Étape 6.3 — Tous les tests

Étape 6.3. Lancer tous les tests.

```
pytest -v
```

Vérification avant de continuer

Dix lignes se terminent par **PASSED**, et la dernière ligne est la suivante.

```
===== 10 passed in 0.03s =====
```

Le tokeniseur appris est complet, ses deux phases sont prouvées.

Étape 6.4 — La mesure sur le corpus réel

Étape 6.4. Mesurer sur le corpus réel (slide 18).

Le script `mesure_fertilite.py` entraîne BPE sur `donnees/entrainement`, du code de la bibliothèque standard de Python et des tickets en français, en anglais et en darija, puis mesure la fertilité sur `donnees/test`, des textes que l'entraînement **n'a jamais vus**.

```
python mesure_fertilite.py
```

Il utilise `entraîner_rapide`, fournie dans `outils.py`, qui donne les mêmes fusions que votre `entraîner` mais tient les comptes de paires à jour au lieu de tout recompter à chaque fusion.

Étape 6.4 — Le tableau attendu

Vérification avant de continuer

corpus d'entraînement : 361875 mots, 7218 mots distincts
7743 fusions apprises en 0.6 s, $|V| = 8000$

$ V $	code	français	anglais	darija
octets	4.00	5.25	5.44	5.41
500	1.86	3.19	2.66	3.63
1000	1.48	2.73	2.16	3.30
2000	1.29	2.38	1.81	3.03
4000	1.15	2.08	1.53	2.52
8000	1.08	1.57	1.24	1.94

liste ordonnée des 1743 fusions du tokeniseur retenu ($|V| = 2000$)
écrite dans merges.json

Étape 6.4 — La lecture du tableau

Le temps affiché varie selon la machine. Les nombres sont exactement ceux de la courbe du **slide 18**.

Un fichier **merges.json** est apparu dans le dossier, c'est **le tokeniseur d'ILIACode**, la liste ordonnée des fusions à $|V| = 2\,000$.

Étape 6.5 — Vos propres chiffres

Étape 6.5. Lire vos chiffres (slides 17 et 18).

Répondez sur votre feuille, avec **vos** nombres.

- 1 Pour le code, de combien la fertilité baisse-t-elle entre 500 et 2 000, puis entre 2 000 et 8 000 ? Où placez-vous le coude ?
- 2 À $|V| = 2\,000$, combien de fois un mot darija coûte-t-il plus de tokens qu'un mot de code ? D'où vient cet écart, d'après le **slide 18** ?

Étape 6.5 — Le ticket au vocabulaire retenu

Relancez python et chargez le tokeniseur retenu.

```
>>> import json
>>> from bpe import encoder, fertilite
>>> fusions = [tuple(p) for p in json.load(open("merges.json"))]
>>> len(fusions)
1743
>>> ticket = "l'app t7bes mnin kanseivi le contrat"
>>> encoder(ticket, fusions)
['l</w>', "'</w>", 'ap', 'p</w>', ' ' </w>', 't', '7', 'b', 'es</w>',
 ' ' </w>', 'm', 'n', 'in</w>', ' ' </w>', 'k', 'an', 'se', 'i', 'v',
 'i</w>', ' ' </w>', 'le</w>', ' ' </w>', 'con', 'tr', 'at</w>']
>>> fertilite(ticket, fusions)
2.625
>>> exit()
```

Étape 6.5 — La lecture du ticket

Le ticket coûte désormais 26 tokens dont 5 espaces, soit 21 tokens pour 8 mots, fertilité 2,6, le chiffre du **slide 17**, contre **49 tokens** avec le mini-corpus de la partie 4. `le</w>` tient en un token, `t7bes` en quatre, `kanseivi` en six. Les mots darija restent chers parce que le corpus d'entraînement n'en contient que quelques centaines.

Partie 6 — Dans une autre application

Dans une autre application

Type d'application. Toute application souveraine en arabe ou en darija, administration, banque, presse.

Comment. Le même script, pointé sur les documents de l'entreprise, donne en une exécution le tableau que le **slide 17** appelle une facture.

Partie 6 — Dans une autre application (suite)

Dans une autre application

La ligne darija y est l'argument qu'une direction comprend, **à nombre de mots égal, trois fois plus de tokens**, donc trois fois la facture, trois fois moins de texte dans la fenêtre et une réponse trois fois plus lente avec un tokeniseur public. Ajouter les tickets de l'entreprise à **donnees/entrainement** et relancer le script abaisse cette ligne, **c'est le travail de la séance 10**.

Les parties du TP

- 1 Partie 0 — La mise en place
- 2 Partie 1 — Les octets
- 3 Partie 2 — Pré-tokeniser
- 4 Partie 3 — Apprendre
- 5 Partie 4 — Encoder
- 6 Partie 5 — Décoder
- 7 Partie 6 — Mesurer
- 8 Partie 7 — Publier**

Lien avec le cours

Cette partie applique le **slide 20** (Le tokeniseur appris est construit). Les trois commandes sont celles du cadre gris du **slide 20**.

Étapes 7.1 et 7.2 — Les fichiers à enregistrer

Étape 7.1. Regarder ce qui a changé.

Vous êtes toujours dans le dossier **seance2**.

```
git status
```

Git affiche en rouge, sous Untracked files, le dossier de la séance.

Étape 7.2. Choisir les fichiers à enregistrer.

```
git add bpe.py test_bpe.py outils.py mesure_fertilite.py merges.json donnees
```

Le fichier **merges.json** est le plus important de la liste, c'est **le tokeniseur lui-même**, versionné comme le **slide 3** l'exige, la séance 3 le chargera tel quel.

Étape 7.3 — Le *commit*

Étape 7.3. Enregistrer une version, le *commit*.

```
git commit -m "Séance 2, le tokeniseur appris, 10 tests verts"
```

Le texte entre guillemets est le message qui décrit la version. Git répond par une ligne qui commence par [main suivie de votre message, puis par le nombre de fichiers enregistrés.

Étape 7.4 — L'envoi sur GitHub

Étape 7.4. Envoyer la version sur GitHub.

```
git push
```

Vérification avant de continuer

La dernière ligne affichée contient `main -> main`. Ouvrez la page de votre dépôt **iliacode** sur GitHub et rafraîchissez-la. Le dossier **seance2** y apparaît, avec votre message de commit à côté.